

Q-Learning for Robotic Grid-World Navigation: From Tabular Methods to Simulated Robot Control

Tanyaradzwa Chisepo

Bethune-Cookman University

Senior Design CS 498

Senior Project

November 28, 2025

Abstract

This project explores the application of classical tabular Q-learning to autonomous robotic navigation in grid-world environments. The research progresses systematically from simple 5×5 grids with fixed start and goal positions to complex 100×100 environments with randomized configurations and obstacles. A key contribution is the development of a 4-tuple relative state representation that encodes both goal and obstacle positions relative to the agent, significantly reducing state space dimensionality while maintaining navigation effectiveness. The study examines the impact of reward shaping, epsilon-decay exploration strategies, and state representation choices on learning stability and policy quality. The learned policies were successfully transferred to a Pioneer P3-DX robot in CoppeliaSim, demonstrating the practical applicability of tabular reinforcement learning methods in simulated robotic systems. Results indicate that while relative state encodings enable generalization across diverse configurations, the exponential growth of state space in larger grids exposes fundamental limitations of tabular methods, suggesting the need for function approximation or hierarchical learning approaches in more complex environments.

Keywords: Q-learning, reinforcement learning, robotic navigation, grid-world, state representation, CoppeliaSim

1. Introduction

1.1 Background on Machine Learning and Reinforcement Learning

In the world of Machine Learning (ML) and Artificial Intelligence (AI), there has been rapid growth in research and applications. Even though machine learning is a subset of AI, the two terms are often used interchangeably. In general, both AI and ML focus on building machines that simulate some form of human intelligence. In simple terms, they aim to teach machines to “think.”

Fundamentally, computers work by taking an input, applying a fixed set of instructions, and producing an output. With the introduction of AI, computers have gained the ability to reason, solve problems, and make decisions. Behind these abilities there is heavy computation on large data sets, which allows systems to learn patterns and make predictions for new inputs that were not seen before [1].

Machine learning is usually divided into several categories, such as supervised learning, unsupervised learning, semi supervised learning, and reinforcement learning. In reinforcement learning, which is the focus of this report, learning is based on reward and penalty. An agent interacts with an environment, takes actions, and receives feedback that helps it learn how to behave over time [2].

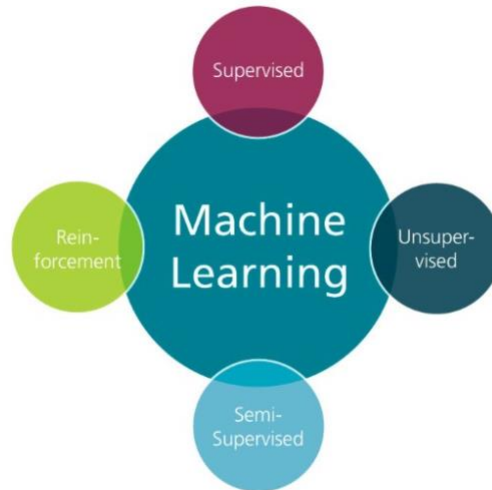


Figure 1: Main Machine Learning Methods

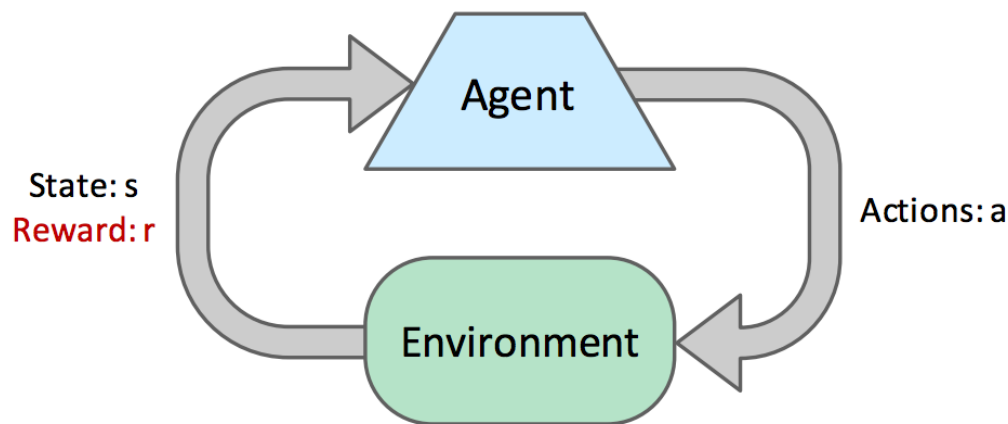


Figure 2: Reinforcement Learning Process

1.2 Q-Learning Fundamentals

Q-learning, an abbreviation for Quality learning, represents a simple, model-free reinforcement learning algorithm where agents learn optimal actions through trial and error, receiving rewards or penalties for their decisions or actions [3]. It does not need a model of the environment and instead learns directly from experience. The learning mechanism can be conceptualized as an agent exploring an environment while maintaining a notebook of action values (the Q-table) which stores the expected utility of each action in every state. The agent updates these values as it explores the environment and uses the table to choose better actions in the future [4]. Over successive iterations, this table converges toward optimal action-value estimates, enabling increasingly effective decision-making.

1.3 Navigation Challenges in Robotics

Navigation capabilities are fundamental to robotic functionality, as autonomous movement toward designated goals supports most practical applications. A robot must be able to move safely and reach a goal in order to perform useful work. Traditional navigation approaches typically rely on complete environmental maps and predetermined paths designed by human operators [5]. While effective in static, well-characterized environments, these methods encounter limitations when facing unknown or partially observable spaces.

Reinforcement learning addresses these challenges by enabling robots to develop navigation policies through experiential learning [6]. When equipped only with local sensory information such as proximity to obstacles and goal direction, robots must learn autonomous navigation policies without step-by-step human guidance [7]. This capability makes Q-learning particularly valuable for tasks including autonomous exploration, dynamic obstacle avoidance, and navigation in environments with incomplete information.

1.4 Grid-World Navigation as a Learning Framework

Grid-world environments provide a discretized representation of space where the environment is partitioned into cells, each representing a distinct state, with agents transitioning between states through discrete actions such as moving up, down, left, or right. This abstraction has become a standard framework in reinforcement learning research due to its balance between simplicity and representational capacity [8].

Grid-worlds facilitate the study of fundamental RL concepts including exploration-exploitation tradeoffs, policy convergence, and value function learning in controlled settings. At each timestep, agents select actions that transition them between cells, with Q-table updates occurring based on received rewards and resulting states. Through repeated interactions, agents develop policies that identify optimal paths to goals while avoiding penalized region, forming a learned policy that guides future decisions [9].

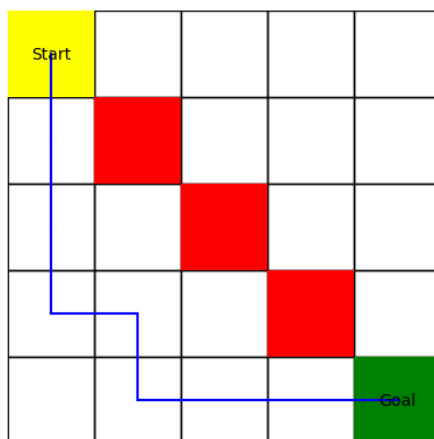


Figure 3: Simple grid-world navigation

1.5 Limitations and Real-World Applicability

Grid-world navigation is widely used in reinforcement learning because it offers a simple and controlled way to test algorithms. The discrete layout makes it easy to study exploration, exploitation, and convergence, and many well-known RL examples such as cliff walking and maze navigation are built around grid-world models [10]. These environments help researchers isolate core behaviors without the complexity of physical systems.

However, grid-worlds differ in several important ways from real environments. Real navigation takes place in continuous space rather than in fixed cells, and robot movement is affected by factors such as wheel slip, uneven surfaces, inertia, and actuator limits. Sensors also produce noisy or incomplete measurements, which changes how the agent perceives its surroundings. Because of these differences, a policy that performs well in a discrete grid does not always transfer directly to a real robot or simulation environment [11].

Even with these limitations, grid-worlds remain valuable for early development and evaluation. They allow researchers to prototype algorithms quickly, study how different state representations affect learning, and identify behaviors before moving on to more complex simulations. In this project, grid-world experiments serve as the foundation for training Q-learning models in controlled settings. Later sections show how these learned policies can be transferred into CoppeliaSim to explore the challenges of bridging abstract models with physically realistic robotic navigation.

2. Problem Statement

Robotic navigation in this project involves training an agent to traverse from start positions to goal locations while avoiding obstacles in grid-based environments. The primary challenge lies in designing a learning system capable of scaling to larger environments without requiring complete environmental maps.

While Q-learning enables learning through trial-and-error exploration, the approach faces a critical limitation: state space dimensionality grows rapidly as grid size increases, making learning inefficient and memory-intensive when states are represented using absolute position coordinates [12].

To address this scalability constraint, the project investigates compact state representations that try to reduce Q-table dimensionality while preserving sufficient information for effective navigation. The exploration progresses from simple absolute coordinate representations through relative goal-based encodings to a final 4-tuple formulation incorporating both goal and obstacle awareness.

3. Goal

The primary goal is to develop and evaluate classical Q-learning approaches for grid-world navigation that scale effectively across environments of increasing complexity, culminating in successful transfer of learned policy to a simulated robotic platform in CoppeliaSim/VRep.

4. Objectives

Objective 1: Establish Foundational Understanding of Reinforcement Learning

Develop comprehensive understanding of Q-learning mechanics, including epsilon-greedy exploration strategies, reward shaping techniques, and basic value update policy.

Objective 2: Implement Grid-World Training Infrastructure

Construct Python-based simulation environments incorporating agent dynamics, goal specifications, obstacle configurations, movement constraints, and reward signal generation to serve as experimental testbeds.

Objective 3: Validate Q-Learning in Simplified Settings

Train and evaluate agents in controlled 5×5 grid environments with fixed start and goal positions and no obstacles to establish baseline performance metrics.

Objective 4: Introduce Environmental Complexity

Systematically add randomization of start/goal positions and obstacle placements across training episodes to promote policy generalization.

Objective 5: Demonstrate Scalability to Larger Environments

Apply 4-tuple relative state representations, reward shaping, step penalties, and epsilon-decay schedules to enable stable learning in grids up to 100×100 cells.

Objective 6: Transfer Policies to Robotic Simulation

Implement learned Q-learning policies within CoppeliaSim's physics-based simulation environment, bridging the gap between discrete grid-world training and continuous robotic navigation.

5. Methodology

5.1 Practical Constraints and Design Challenges

Several constraints shaped the experimental approach:

1. **State Space Growth:** State dimensionality increases exponentially with grid size, particularly when using absolute coordinate representations, necessitating more compact state encodings for larger environments.
2. **Incomplete Information:** Agents operate without global maps, relying solely on local observations to develop navigation policies.
3. **Training Instability:** Large state spaces exhibit learning instability, especially with poorly designed reward structures that fail to provide adequate guidance signals.
4. **Exploration-Exploitation Balance:** Simple reward structures prove insufficient in complex environments, where agents may converge to suboptimal policies, exhibit cyclic behaviors, or fail to make progress toward goals.

These constraints motivated a staged development approach, beginning with minimal grid configurations and gradually introducing complexity through careful adjustments to state representation, reward shaping, and exploration schedules.

5.2 Design Standards and Robustness Criteria

To ensure stable and reproducible learning, the following design standards were established:

- **Incremental Complexity:** Initial training phases utilized small grids (5×5) with fixed start and goal positions to minimize environmental variance and facilitate debugging.
- **Systematic Feature Introduction:** Obstacles were introduced only after agents demonstrated reliable performance in obstacle-free environments.
- **State Representation Evolution:** The transition from absolute coordinates to 4-tuple relative representations was motivated by the need to control Q-table growth and improve generalization.
- **Reward Shaping:** Rewards were carefully designed to provide directional guidance toward goals while penalizing unproductive behaviors and collisions.
- **Adaptive Exploration:** Epsilon-decay schedules enabled broad initial exploration followed by gradual refinement toward exploitation of discovered strategies.

The "factor of safety" in this context refers to maintaining sufficient training signal strength and exploration duration to prevent premature convergence to local optima or degenerate policies characterized by repetitive, non-productive behavior patterns.

5.3 Project Timeline and Development Phases

The project proceeded through six distinct phases:

Phase 1 (Week 1-2): Basic Q-learning implementation and validation in 5×5 grids with fixed configurations and no obstacles.

Phase 2 (Weeks 3-4): Introduction of randomized start/goal positions and single obstacle placement.

Phase 3 (Weeks 4-5): Development and testing of relative state representations (goal-relative encoding).

Phase 4 (Weeks 6-7): Reward shaping refinement and epsilon-decay schedule optimization.

Phase 5 (Weeks 7-9): Scaling experiments across 10×10, 50×50, and 100×100 grid environments using 4-tuple state representation.

Phase 6 (Week 10): CoppeliaSim integration, policy transfer, and robotic simulation testing.

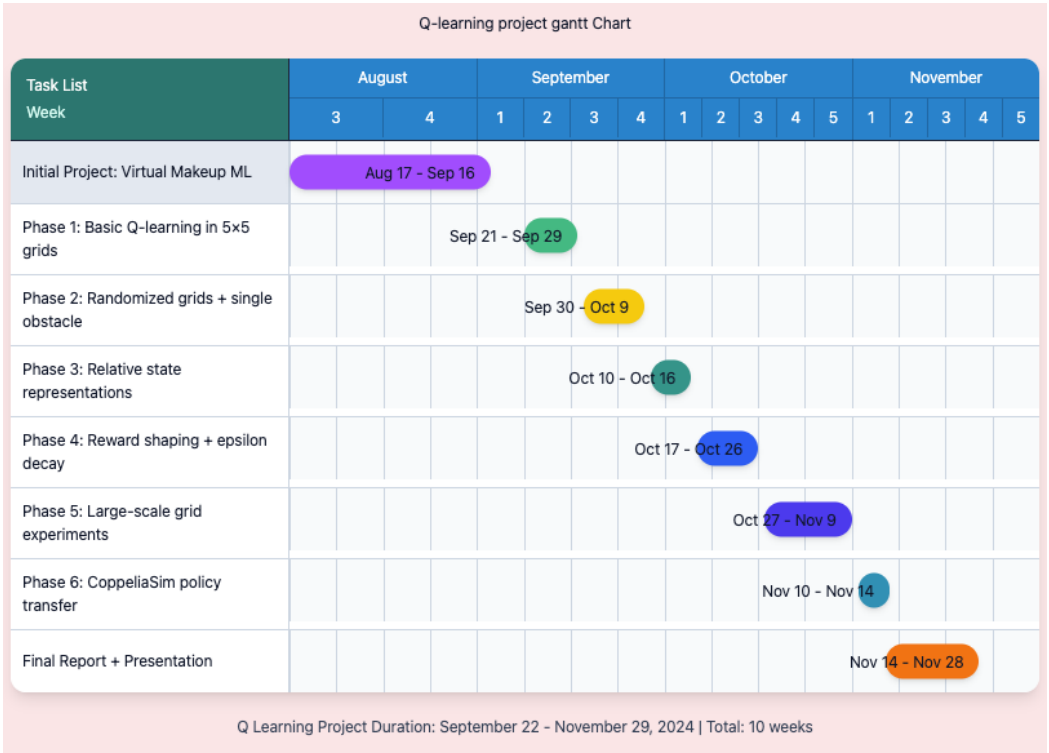


Figure 4: Project Gantt Chart

6. Development

This section explains how the project was built from the ground up, starting with a simple grid based Q learning algorithm and gradually adding complexity. The development process was exploratory, with many small experiments carried out to understand what worked, what failed, and what needed to change for a generalized approach that could be applied to larger environments.

6.1 Baseline Implementation: 5×5 Grid Without Obstacles

The initial prototype established fundamental Q-learning functionality in a minimal environment to verify algorithmic correctness before introducing additional complexity.

6.1.1 Environment Configuration:

- Grid dimensions: 5×5 (25 discrete states)
- Agent start position: (0,0)
- Goal position: (4,4)
- Action space: {up, down, left, right}
- Reward structure: +100 for goal achievement, -1 per step
- Training duration: 100 episodes
- Q-table shape: 25 states $\times$ 4 actions

6.1.2 Implementation Details:

The baseline system implemented essential components that would form the foundation for all subsequent versions:

- `state_to_index()` and `index_to_state()`: Bidirectional mapping between 2D grid coordinates and linear state indices
- `choose_action()`: Epsilon-greedy action selection balancing exploration and exploitation
- `step_env()`: Environment transition function computing next states and rewards
- `render_path()`: Visualization utility for trajectory analysis
- Standard Q-learning update equation
- Logging utilities and plots for episode visualizations

6.1.3 Code Walkthrough

Environment Setup

```
n_rows, n_cols = 5, 5
start = (0, 0)
goal = (4, 4)

ACTIONS = [(-1,0), (1,0), (0,-1), (0,1)]
n_actions = len(ACTIONS)
n_states = n_rows * n_cols

alpha = 0.1
gamma = 0.90
epsilon = 0.20
episodes = 100
max_steps = 20

Q = np.zeros((n_states, n_actions), dtype=np.float32)
```

Explanation:

This initializes the grid size, start and goal, the four movement actions, and all training hyperparameters.

A 25×4 Q-table is created, one row per state, one column per action.

State Mapping Functions

```
def state_to_index(state):
    return state[0] * n_cols + state[1]

def index_to_state(idx):
    return (idx // n_cols, idx % n_cols)
```

Explanation:

These functions convert between (row, col) coordinates and a single state index. This allows the table to store values for each cell without handling 2D arrays.

Environment Transition

```
def is_valid(state):
    r, c = state
    return (0 <= r < n_rows) and (0 <= c < n_cols)

def step_env(state, action):
    r, c = state
    dr, dc = ACTIONS[action]
    next_state = (r + dr, c + dc)

    if not is_valid(next_state):
        next_state = state

    reward = -1.0
    done = False

    if next_state == goal:
        reward = 100.0
        done = True

    return next_state, reward, done
```

Explanation:

The agent tries to move up/down/left/right. If the move goes off the grid, the agent stays still. Each step gives -1 reward, while reaching the goal gives $+100$ and ends the episode.

Action Selection (ϵ -greedy)

```
def choose_action(state_idx, eps):
    if np.random.rand() < eps:
        return np.random.randint(n_actions)
    return int(np.argmax(Q[state_idx]))
```

Explanation:

The agent explores with probability ϵ and exploits the learned Q-values otherwise. In this baseline version ϵ stays constant, so exploration is stable across all episodes.

Path Rendering

```
def render_path(path, title="Episode Trajectory"):
    grid = np.zeros((n_rows, n_cols, 3), dtype=np.uint8) + 255

    grid[start] = [0,0,255]
    grid[goal] = [255,0,0]

    for (r,c) in path:
        if (r,c) not in (start,goal):
            grid[r,c] = [0,255,0]

    plt.figure()
    plt.imshow(grid)
    plt.title(title)
    plt.axis("off")
    plt.show()
```

Explanation:

The start cell is marked in blue, the goal in red, and the agent's visited cells in green. This tool helps visualize whether learning is improving over time.

Main Training Loop

```
steps_per_episode = []
reward_per_episode = []

for ep in range(1, episodes+1):
    state = start
    state_idx = state_to_index(state)
    done = False
    steps = 0
    total_reward = 0.0
    path = [state]

    while not done and steps < max_steps:
        action = choose_action(state_idx, epsilon)
        next_state, reward, done = step_env(state, action)
        next_idx = state_to_index(next_state)

        best_next = 0.0 if done else np.max(Q[next_idx])
        td_target = reward + gamma * best_next
        td_error = td_target - Q[state_idx, action]
        Q[state_idx, action] += alpha * td_error

        state, state_idx = next_state, next_idx
        steps += 1
        total_reward += reward
        path.append(state)

    steps_per_episode.append(steps)
    reward_per_episode.append(total_reward)

    if ep % 100 == 0:
        render_path(path, f"Episode {ep}")
```

Explanation:

Each episode runs until the agent reaches the goal or hits the step limit.
 The Q-table is updated using the standard temporal difference rule.
 Every 100 episodes the script visualizes the full trajectory.

Plotting Learning Curves

```
plt.figure()
plt.plot(range(1, episodes+1), steps_per_episode)
plt.title("Steps per Episode")

plt.figure()
plt.plot(range(1, episodes+1), reward_per_episode)
plt.title("Reward per Episode")
```

Explanation:

The plots show the improvement in path efficiency and rewards across episodes.
 The expected trend is fewer steps and higher rewards as the agent learns.

Greedy Policy Evaluation

```
for run in range(1,6):
    state = start
    steps = 0
    total_reward = 0.0

    while state != goal and steps < max_steps:
        s_idx = state_to_index(state)
        action = int(np.argmax(Q[s_idx]))
        next_state, reward, done = step_env(state, action)
        state = next_state
        steps += 1
        total_reward += reward

    print("Run", run, "steps =", steps, "reward =", total_reward)
```

Explanation:

After training, the agent switches to pure exploitation ($\epsilon=0$).
 The learned policy is then tested for five runs to check consistency and stability.

6.1.4 Q-update formula

The project relies on tabular Q-learning, where the value of choosing action a in state s is updated according to the classical Bellman update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

This rule incrementally pushes the Q-value toward the observed reward plus the discounted estimate of the best achievable future value.

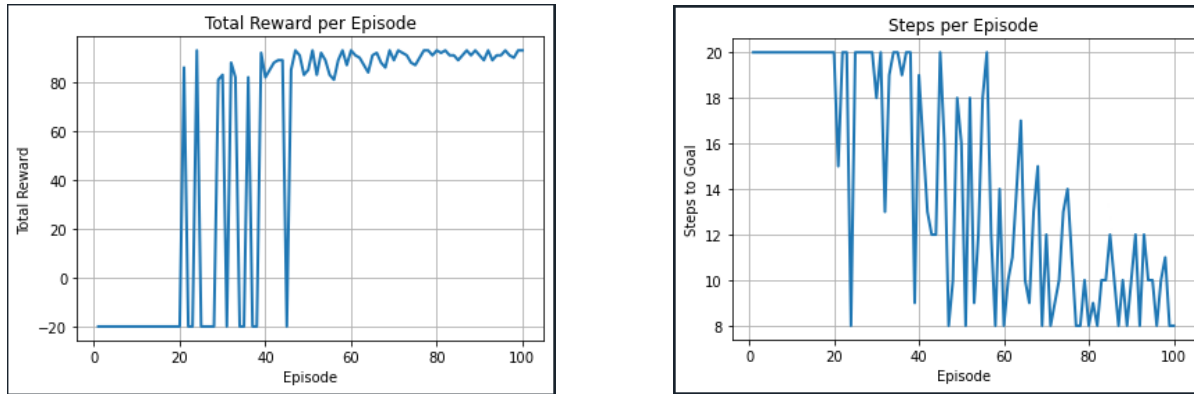


Figure 5: Sample Plots for cumulative reward and steps per episode visualization

6.1.5 Training Behavior:

Initial episodes exhibited expected exploratory behavior with lengthy, meandering paths. As training progressed:

- Path lengths decreased consistently
- Total episode rewards increased steadily
- Final episodes demonstrated near-optimal diagonal trajectories toward the goal
- Learning curves displayed smooth convergence without significant oscillations

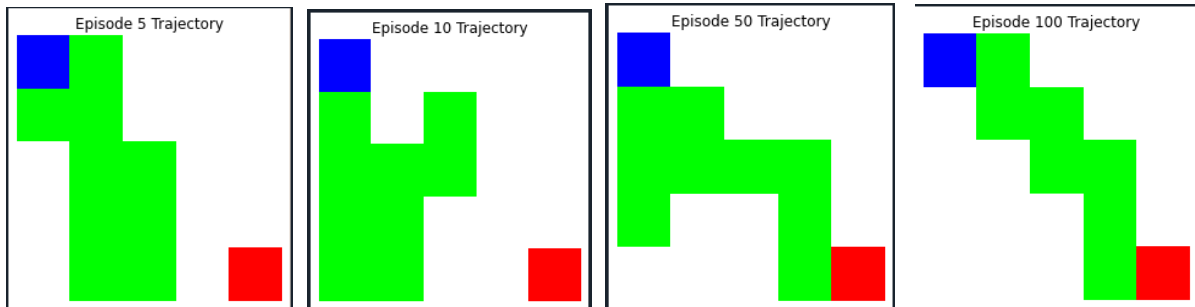


Figure 6: Sample trajectory plots for baseline

6.1.6 Policy Evaluation:

Following training, the learned policy was evaluated through five greedy test runs ($\epsilon=0$, no exploration). All test runs successfully reached the goal with consistent, near-optimal path lengths, confirming Q-table convergence.

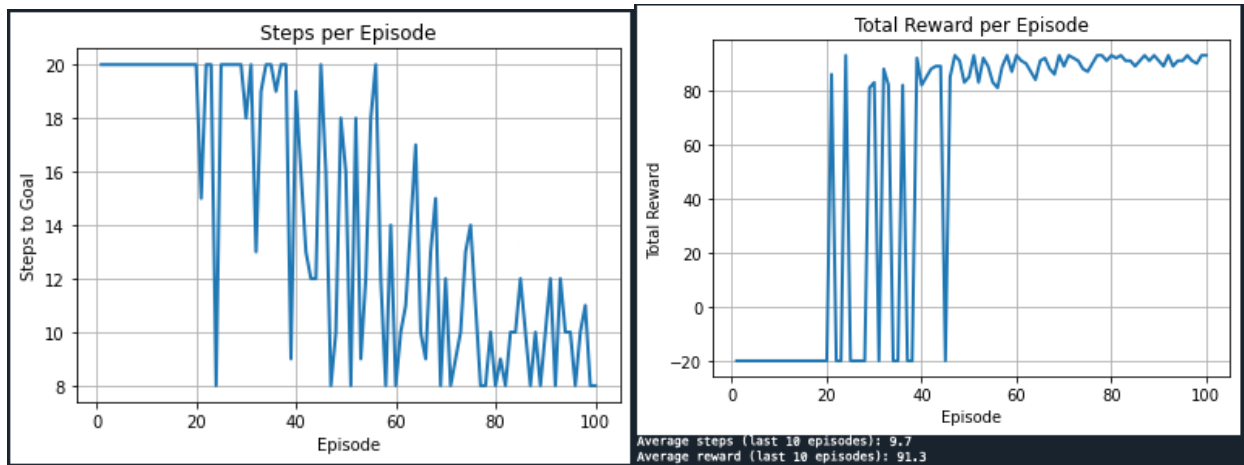


Figure 7: Baseline result trends

```
Testing learned policy (5 runs, greedy actions):  
Run 1: steps = 8, total_reward = 93.0, reached_goal = True  
Run 2: steps = 8, total_reward = 93.0, reached_goal = True  
Run 3: steps = 8, total_reward = 93.0, reached_goal = True  
Run 4: steps = 8, total_reward = 93.0, reached_goal = True  
Run 5: steps = 8, total_reward = 93.0, reached_goal = True
```

Figure 8: Results for 5 test runs on baseline

	0	1	2	3
0	-2.66656	-2.60655	-2.63909	1.4654
1	-2.04512	7.80205	-2.20045	-1.75379
2	-1.39942	3.93288	-1.45193	-1.30596
3	-0.853735	0.176702	-0.814971	-0.817301
4	-0.481367	-0.549271	-0.563508	-0.4901
5	-2.075	-1.98945	-2.03457	-0.948801
6	-1.28005	-1.49693	-1.59107	18.5572
7	-1.11274	34.2617	-0.449107	1.5058
8	-0.5927	30.4769	-0.671935	-0.607474
9	-0.505104	-0.3691	-0.443661	-0.392509
10	-1.54022	-1.41554	-1.47219	-1.4369
11	-1.14773	-0.995146	-1.37806	4.3396
12	1.27968	1.05154	-0.679685	52.599
13	5.5244	71.5425	1.58319	2.81734
14	-0.199	18.7478	0.998331	-0.199
15	-0.977151	-0.93774	-0.983933	0.0501648
16	-0.646374	5.60586	-0.72897	-0.0183542
17	-0.421935	2.45114	-0.220924	28.759
18	6.18603	87.1018	3.50871	9.16575
19	0.255823	61.258	5.33728	0.8
20	-0.725196	-0.856439	-0.679347	-0.403016
21	-0.619406	-0.577864	-0.628484	23.5497
22	-0.102839	-0.29701	-0.31385	58.4862
23	9.10211	16.549	4.56726	99.8003
24	0	0	0	0

Figure 9: Full 25-State Q-table with updated values

Significance:

This baseline established that:

1. Q-table updates function correctly
2. Policy improvement occurs even without epsilon decay in simple deterministic environments
3. Reward shaping effectively guides agents toward shortest paths
4. Visualization tools accurately reflect learning progress
5. The implementation provides a solid foundation for complexity scaling

6.2 Introducing Single Obstacle with Penalties

Following successful baseline validation, environmental complexity was increased by introducing a single obstacle at position (2,2) with a -5 penalty for collision attempts.

Modified Configuration:

- Obstacle position: (2,2) - center of 5×5 grid
- Collision penalty: -5
- All other parameters unchanged from Section 6.1

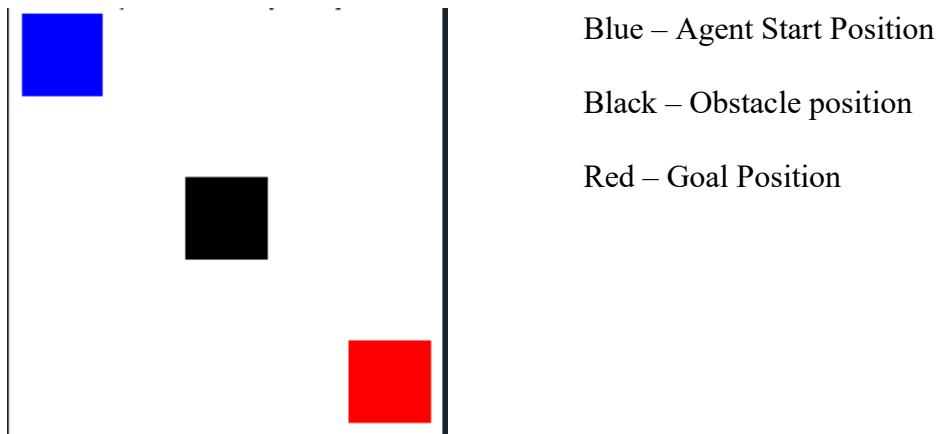


Figure 10: Grid World Visualization

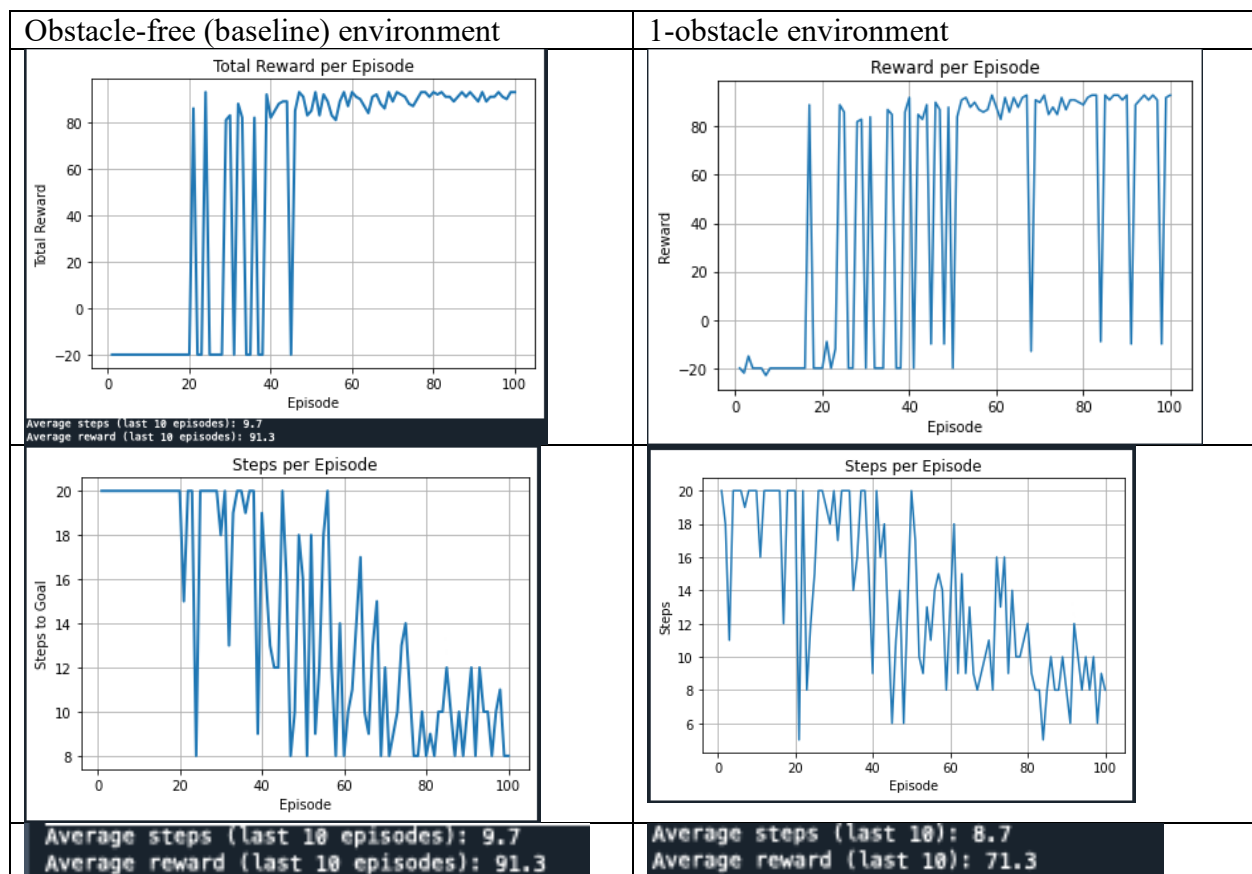
Two collision-handling modes were tested:

1. **Continuation mode:** Episodes proceed after collisions with agent remaining in pre-collision position
2. **Termination mode:** Episodes end immediately upon collision

Training Dynamics:

Early training episodes exhibited frequent obstacle collisions and circuitous paths as agents learned avoidance strategies. With extended training:

- Collision frequency decreased substantially
- Agents developed consistent detour strategies around the obstacle
- Total episode rewards improved despite additional path length requirements
- Learning curves demonstrated more gradual convergence compared to obstacle-free environments



Performance Analysis:

Compared to the baseline scenario, the introduction of a single obstacle resulted in:

- Average final-episode rewards decreased by approximately 20 points
- Mean path lengths increased due to necessary detours
- Greater training variance during early episodes
- More diverse optimal policies depending on obstacle-avoidance strategies

Greedy policy testing across five runs demonstrated reliable goal achievement with consistent obstacle avoidance, validating that the learned policy incorporated collision aversion into navigation strategies.

Key Insights:

This experiment revealed Q-learning's sensitivity to reward structure design. Even a small penalty (-5) proved sufficient to reshape policies toward safer trajectories. The Q-table developed clear value gradients pushing actions away from the obstacle region while maintaining goal-directed behavior.

The added obstacle introduces a negative reinforcement signal that alters the learned policy. Whenever the agent attempts to step into the blocked cell, a penalty is applied and the movement is cancelled.

This encourages paths that avoid the obstacle and produces longer but safer trajectories. In early episodes the agent often collides with the obstacle, but as training progresses the Q-values associated with those actions drop, and the agent learns to route itself around the blocked region.

Code Walkthrough

Obstacle definition (environment setup)

```
# One obstacle
obstacle = (2, 2)
obstacles = {obstacle}

OBSTACLE_PENALTY = -5.0
END_EPISODE_ON_OBSTACLE = True
```

Explanation:

The obstacle is placed at cell (2,2).

A fixed penalty of -5 is applied whenever the agent attempts to move into this cell.

If `END_EPISODE_ON_OBSTACLE` is enabled, an obstacle collision immediately ends the episode.

Collision and penalty handling in the environment transition

```
def step_env(state, action):
    r, c = state
    dr, dc = ACTIONS[action]
    nxt = (r + dr, c + dc)

    # Outside grid
    if not inside_grid(nxt):
        return state, -1.0, False

    # Obstacle hit
    if nxt in obstacles:
        reward = OBSTACLE_PENALTY
        done = END_EPISODE_ON_OBSTACLE
```

```

        return state, reward, done

    # Normal movement
    reward = -1.0
    done = False
    if nxt == goal:
        reward = 100.0
        done = True

    return nxt, reward, done

```

Explanation:

Before applying a normal move, the function checks whether the next cell corresponds to the obstacle.

If the agent attempts to move into it:

- The agent is forced to stay in the same position.
- A penalty is applied.
- The episode ends if the setting requires it.

This simple branching prevents the agent from occupying the obstacle cell and teaches the Q-learner that this move is harmful.

Visualization marking the obstacle

```

def render_path(path, title="Episode Path"):
    grid = np.zeros((n_rows, n_cols, 3), dtype=np.uint8) + 255

    # obstacle in black
    for (r, c) in obstacles:
        grid[r, c] = [0, 0, 0]

    grid[start] = [0, 0, 255]
    grid[goal] = [255, 0, 0]

    for (r, c) in path:
        if (r, c) not in obstacles and (r, c) not in [start, goal]:
            grid[r, c] = [0, 255, 0]

    plt.figure()
    plt.imshow(grid)
    plt.title(title)
    plt.axis("off")
    plt.show()

```

Explanation:

The plot colors the obstacle cell in black.

This makes it easy to visually confirm whether the policy still tries to approach or collide with it during training.

6.3 Randomization Challenges: Absolute States with Variable Configurations

To assess policy generalization, start and goal positions were randomized across episodes while maintaining absolute coordinate state representation.

Configuration Changes:

- Start position: Random cell per episode
- Goal position: Random cell per episode (distinct from start)
- Obstacle: Removed for this phase
- State representation: Absolute (row, col) coordinates
- Q-table: 25 states $\times$ 4 actions (unchanged)

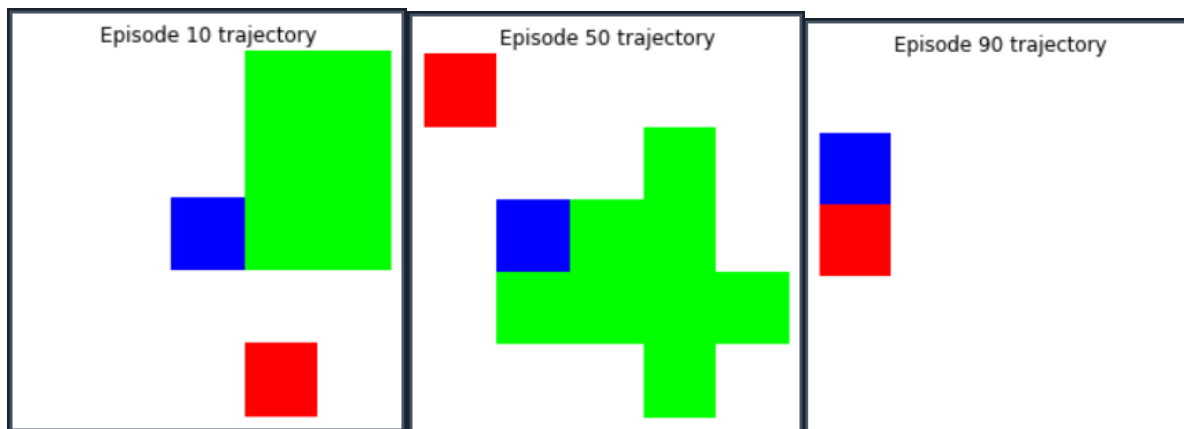


Figure 11: Sample random grid configurations

Observed Instability:

This configuration revealed a critical limitation: absolute coordinate representations lack semantic consistency across configurations. A state such as (2,3) could represent:

- Near-goal position in one episode
- Far-from-goal position in another episode
- Goal-left configuration versus goal-right configuration

This semantic inconsistency prevented Q-value stabilization, as updates for identical state indices reflected contradictory experiences across episodes.

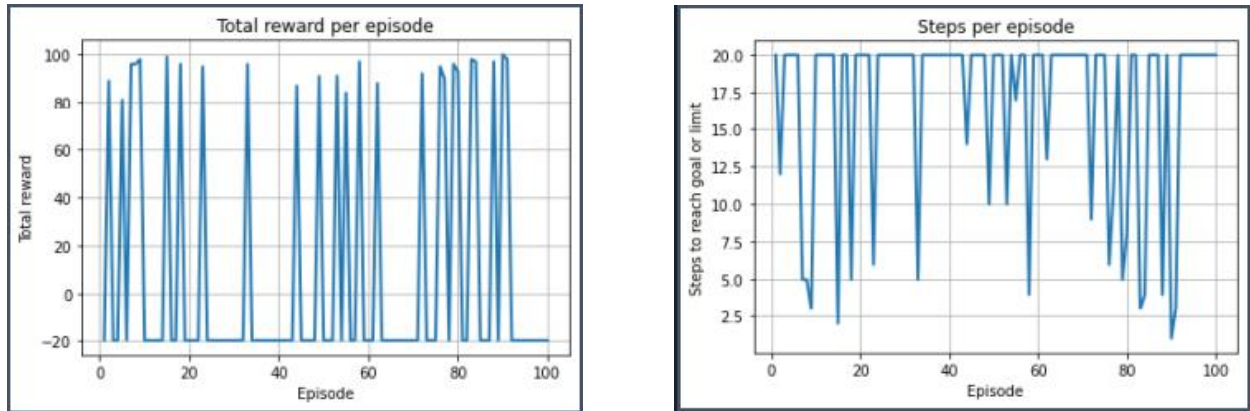


Figure 12: The reward and steps plots show no convergence

Manifestations of Learning Failure:

- Episode rewards oscillated without converging
- Step counts showed no consistent decreasing trend
- Consecutive episodes with identical start-goal configurations produced different policies
- Trajectory visualizations revealed incoherent, sometimes circular paths
- Q-values continuously overwrote previous learning without consolidation

Greedy Policy Test Results (5 Runs)

Table 1: Random configuration evaluation results

Run	Start	Goal	Final Position	Steps	Total Reward	Reached Goal
1	(1, 3)	(3, 4)	(1, 3)	20	-20.0	False
2	(2, 1)	(2, 4)	(2, 3)	20	-20.0	False
3	(0, 3)	(1, 0)	(2, 3)	20	-20.0	False
4	(0, 1)	(0, 3)	(0, 3)	2	99.0	True
5	(3, 3)	(4, 0)	(1, 3)	20	-20.0	False

Root Cause Analysis:

The fundamental issue stems from state representation: absolute positions provide no information about goal location or direction. Without goal-relative information, agents cannot develop transferable policies that generalize across different spatial configurations. Each unique start-goal pair effectively represents a distinct navigation problem, but the state encoding treats all positions with identical coordinates as equivalent regardless of goal position.

This realization directly motivated the state representation redesign implemented in Section 6.4.

6.4 Relative State Encoding: Goal-Centric Representation

Relative encoding was introduced to fix the instability found with absolute cell states in Section 6.3. Instead of storing “where the agent is on the grid,” the state now stores “where the goal is relative to the agent.” This gives each state a clear and reusable meaning across different start and goal locations.

State Representation Redesign:

The new state is defined using goal-relative offsets:

$$\Delta x = agent_row - goal_row$$

$$\Delta y = agent_col - goal_col$$

With this definition, a state such as ($\Delta x = 2, \Delta y = 1$) always means “the agent is two cells below and one cell to the right of the goal,” regardless of the absolute grid coordinates. The same relative pattern appears across episodes and can be reused by the Q-table.

Implementation details

For the 5×5 grid:

- Agent and goal rows are in $[0, 4]$
- Agent and goal columns are in $[0, 4]$
- Δx and Δy are in $[-4, +4]$
- The shifted index range is $[0, 8]$ in each dimension
- Total relative states: $9 \times 9 = 81$

The code uses two short helper functions to implement this mapping.

In the actual implementation this logic appears as:

```
def relative_offset(agent, goal):
    dr = agent[0] - goal[0]
    dc = agent[1] - goal[1]
    return dr, dc

def rel_state_to_index(agent, goal):
    dr, dc = relative_offset(agent, goal)
    shift_r = dr + (n_rows - 1)    # shift from [-4,4] to [0,8]
    shift_c = dc + (n_cols - 1)
    return shift_r * rel_cols + shift_c
```

`relative_offset` computes the signed difference between agent and goal.

`rel_state_to_index` then shifts these values into a non-negative range and flattens the 2D relative grid into a single Q-table row index. The Q-update remains unchanged, but it now operates over this relative index instead of an absolute cell index.

Training Improvements:

Once the state carried pure goal-relative information, the learning curves changed in a clear way:

- Steps per episode started high but decreased steadily over time
- Episode rewards, when smoothed, increased in a more monotonic way
- The learned paths became straight or gently curved toward the goal instead of wandering
- Policy behavior across different start and goal pairs looked similar and predictable

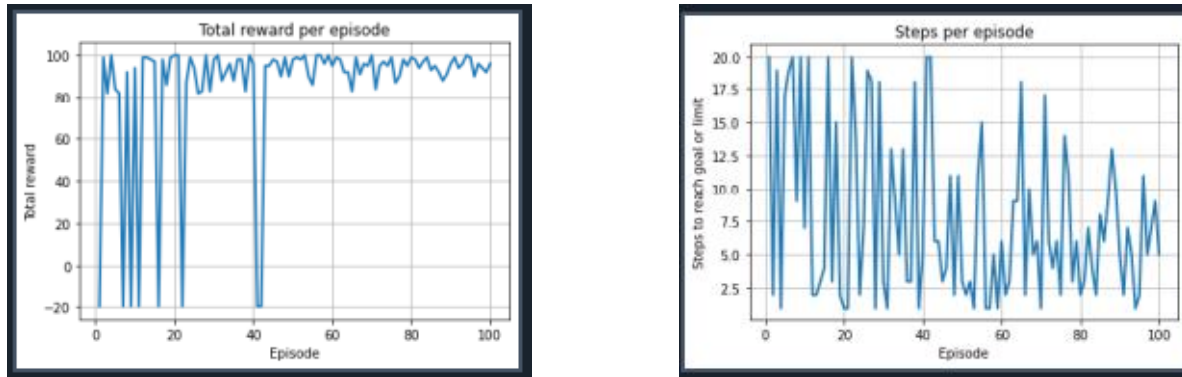


Figure 13: Plots for relative state encoding

Greedy Policy Evaluation:

Testing with pure exploitation (ϵ -greedy) demonstrated that agents reliably moved directly toward goals or selected appropriate detours when necessary. Trajectory visualizations confirmed goal-directed behavior across diverse start-goal configurations.

Greedy Policy Evaluation (5 Random Runs)

Table 2

Run	Start	Goal	Final Position	Steps	Total Reward	Reached Goal
1	(0,1)	(1,1)	(1,1)	1	100.0	True
2	(3,2)	(2,0)	(2,0)	3	98.0	True
3	(1,3)	(2,1)	(2,1)	3	98.0	True
4	(2,2)	(0,3)	(0,3)	3	98.0	True
5	(3,2)	(1,3)	(1,3)	3	98.0	True

Key Insight:

The dramatic performance improvement stemmed entirely from state representation redesign, not from changes to rewards, learning rates, or exploration strategies. Once agents received goal-relative information, the learning problem became well-defined and policies converged to effective solutions.

This success validated the principle that state representations must capture task-relevant information – in this case, directional relationships to objectives rather than absolute grid cell positions.

6. 5.1 Advanced 5×5 Model: Integrated Randomization with Obstacle Awareness

This section studies how relative state encoding interacts with random start and goal configurations when a single obstacle and penalty are present. All V5.x variants share the same core design:

- 5×5 grid world
- One fixed obstacle at (2,2)
- Random start and goal each episode, never on the obstacle
- Relative state representation, agent position encoded with respect to the goal
- Obstacle penalty `OBSTACLE_PENALTY = -5.0`
- Step cost of `-1` and goal reward of `+100`
- Tabular Q learning over 81 relative states and 4 actions



Figure 14: Random goal/start, fixed obstacle

6.5.1 Version 5.1: Fixed Exploration Rate

Version 5.1 extends the relative state encoding from Section 6.4 by adding a single obstacle and an associated penalty. The environment keeps a random start and goal each episode, but any sampled start or goal on the obstacle is rejected.

Configuration:

- Grid size: 5×5
- Obstacle: (2, 2)
- Relative state space: 81 states ($\Delta r, \Delta c$ in $[-4, 4]$)
- Epsilon: fixed at 0.20 for the whole run
- Episodes: typically 1 000, with later experiments up to 1 000 000
- Obstacle penalty: -5.0 reward on attempted moves into (2, 2)

The relative state mapping is implemented as:

```
def relative_offset(agent, goal):
    dr = agent[0] - goal[0]
    dc = agent[1] - goal[1]
    return dr, dc

def rel_state_to_index(agent, goal):
    dr, dc = relative_offset(agent, goal)
    shift_r = dr + (n_rows - 1)    # shift into [0, 8]
    shift_c = dc + (n_cols - 1)
    return shift_r * rel_cols + shift_c
```

Obstacle handling and penalties are injected inside the transition function:

```
def step_env(agent_state, action, goal):
    r, c = agent_state
    dr, dc = ACTIONS[action]
    nxt = (r + dr, c + dc)

    if not inside_grid(nxt):
        return agent_state, -1.0, False

    if nxt in obstacles:
        reward = OBSTACLE_PENALTY
        done = END_EPISODE_ON_OBSTACLE
        return agent_state, reward, done

    reward = -1.0
    done = False
    if nxt == goal:
        reward = 100.0
        done = True

    return nxt, reward, done
```

The Q table is updated in the relative state space:

```
best_next = 0.0 if done else np.max(Q[next_rel_idx])
td_target = reward + gamma * best_next
td_error = td_target - Q[rel_idx, action]
Q[rel_idx, action] += alpha * td_error
```

Observed behaviour for Version 5.1:

- The agent gradually learns to move toward the goal while avoiding the obstacle in many cases
- However, because epsilon remains fixed at 0.20, exploration noise persists indefinitely
- Learning curves improve at first, then become noisy and unstable for long runs
- Extending training to several hundred thousand episodes does not guarantee better policies, and in some trials performance degrades after about 500 000 episodes
- The Q table never fully settles, and useful Q entries can be corrupted by ongoing random actions

These experiments confirm that additional training alone is not sufficient when epsilon is fixed. For random start and goal positions, the agent keeps revisiting many states with a 20 percent chance of taking random actions, which blocks stable convergence.

6.5.2 Version 5.2: Structured Epsilon Decay (80/20 Schedule)

Version 5.2 keeps the same relative representation and obstacle configuration but introduces a structured exploration schedule that decays epsilon over time and allocates a final exploitation phase.

Epsilon schedule:

- Episodes 0 to 80 percent of total: epsilon decays linearly from 0.20 to 0.00
- Episodes 80 percent to 100 percent: epsilon is fixed at 0.00 so the agent performs pure greedy exploitation and consolidates the learned policy

A simple schedule can be implemented as:

```
def epsilon_schedule(ep, total_episodes, eps_start, eps_end, explore_frac):
    t = ep / float(total_episodes)
    if t <= explore_frac:
        ratio = t / explore_frac
        return eps_start + (eps_end - eps_start) * ratio
    else:
        return eps_end
```

During training the current epsilon is computed once per episode:

```

eps = epsilon_schedule(ep, episodes, epsilon_start, epsilon_end,
explore_fraction)
epsilon_history.append(eps)
action = choose_action(rel_idx, eps)

```

The rest of the loop uses the same relative state logic and obstacle penalty as in Version 5.1:

```

next_agent_state, reward, done = step_env(agent_state, action, goal)
next_rel_idx = rel_state_to_index(next_agent_state, goal)

best_next = 0.0 if done else np.max(Q[next_rel_idx])
td_target = reward + gamma * best_next
td_error = td_target - Q[rel_idx, action]
Q[rel_idx, action] += alpha * td_error

```

Relative to V5.1, V5.2 shows:

- Smoother reward curves that rise steadily and remain stable even at 1 000 000 episodes
- Step counts that decrease consistently, with no late stage regression
- Greedy test runs that show reliable obstacle avoidance and direct goal seeking
- Less variance across independent runs, because exploration noise is gradually removed

Table: Performance Comparison of Fixed ϵ (V5.1) vs Decaying ϵ (V5.2)

Table 3

Episodes	V5.1 ($\epsilon = 0.20$)	V5.2 (ϵ decay)	Improvement
100	~45	~60	+33%
1,000	~55	~80	+45%
50,000	~70	~92	+31%
100,000	~72	~95	+32%
500,000	~94	~98	+4%
1,000,000	~75	~96	+28%

Key findings from Versions 5.1 to 5.2:

1. Random start and goal configurations benefit from relative state encoding but still require careful control of exploration.
2. Obstacle penalties are effective only when the Q table can converge, which fixed epsilon does not allow at very long horizons.
3. Fixed exploration rates support early learning but harm long term stability.

4. Structured epsilon decay that transitions from exploration to exploitation greatly improves convergence speed and final policy quality.

Version 5.2 therefore provides the strongest and most stable performance and serves as a practical template for training Q learning agents in larger grids with random start and goal locations and embedded obstacles.

6.6 Scaling to Large Environments: 4-Tuple Relative State Representation

As experiments progressed toward larger grids, the limitation of including only goal information in the state representation became apparent. To enable obstacle-aware navigation in complex environments, the state encoding was expanded to a 4-tuple format.

6.6.1 Motivation for 4-Tuple States

Early attempts used absolute coordinates for the agent, goal, and obstacle. For an $n \times n$ grid this produces on the order of n^6 distinct states, which becomes infeasible even for modest grid sizes because tabular Q-learning must maintain a separate value for each state action pair [12].

To avoid this explosion while still exposing both goal and obstacle information, the state was redesigned as a 4-tuple of *relative* offsets from the agent to the goal and to the obstacle.

6.6.2 State Representation Design

The new state is:

$$s = (\Delta x_g, \Delta y_g, \Delta x_o, \Delta y_o)$$

where

- $\Delta x_g = \text{goal_row} - \text{agent_row}$
- $\Delta y_g = \text{goal_col} - \text{agent_col}$
- $\Delta x_o = \text{obst_row} - \text{agent_row}$
- $\Delta y_o = \text{obst_col} - \text{agent_col}$

For an $n \times n$ grid each delta lies in $[-(n - 1), n - 1]$. The implementation defines

```
DX_MIN, DX_MAX = -(n_rows - 1), (n_rows - 1)
DY_MIN, DY_MAX = -(n_cols - 1), (n_cols - 1)
D = DX_MAX - DX_MIN + 1          # number of possible values per delta

# total number of 4-tuple states
N_STATES = D * D * D * D
```

For an $n \times n$ grid:

- each delta component ranges over $[-(n - 1), n - 1]$
- range size is $D = 2(n - 1) + 1$
- total number of states is D^4

Example sizes:

- 10×10 grid: $D = 19$, states = 130,321
- 50×50 grid: $D = 99$, states $\approx$ 96 million
- 100×100 grid: $D = 199$, states $\approx$ 1.57 billion

The encoding function mirrors this design:

```
def rel_components(agent, goal, obstacle):
    ar, ac = agent
    gr, gc = goal
    or_, oc = obstacle
    return (gr - ar, gc - ac, or_ - ar, oc - ac)

def encode4(agent, goal, obstacle):
    dxg, dyg, dxo, dyo = rel_components(agent, goal, obstacle)

    ixg = dxg - DX_MIN
    iyg = dyg - DY_MIN
    ixo = dxo - DX_MIN
    iyo = dyo - DY_MIN

    return ((ixg * D + iyg) * D + ixo) * D + iyo
```

This encoding keeps grid symmetry: any two configurations that share the same relative geometry produce the same state index, regardless of absolute location. This is closely related to goal conditioned value functions and universal value function approximators that treat the goal as part of the input representation [13].

6.6.3 Environment and Reward Structure

Configuration

- Grid sizes tested: 10×10, 20×20, 50×50, 100×100
- One obstacle per episode (random position)
- Random start, goal, and obstacle, all distinct
- Maximum episode length: 50 steps
- Action space: {up, down, left, right}

Euclidean Distance Based Reward Shaping

Rewards are based on Euclidean distance to the goal:

- goal reached: +100 and terminate

- invalid move (outside grid or into obstacle): reward = -1, position unchanged
- valid move:
 - if distance to goal decreases: +1
 - otherwise: -1

```
def euclid(a, b):
    return np.hypot(a[0] - b[0], a[1] - b[1])

# inside training loop
if next_state == goal:
    reward = 100.0
    done = True
else:
    if not moved:
        reward = -1.0
    else:
        d_old = euclid(state, goal)
        d_new = euclid(next_state, goal)
        reward = +1.0 if d_new < d_old else -1.0
```

This potential based shaping encourages progress toward the goal and penalizes collisions and aimless wandering without changing the set of optimal policies when designed correctly [14].

6.6.4 Epsilon Schedule and Training Procedure

The epsilon schedule uses an exponential decay that depends on the total number of episodes:

```
eps, eps_min = 1.0, 0.1
eps_decay = (1 - (1 / (episodes * 0.4)))

# inside episode loop
epsilon_history.append(eps)
...
a = choose_action(s_idx, eps)
...
eps = max(eps_min, eps * eps_decay)
```

Epsilon starts at 1.0, decays aggressively during roughly the first 40% of training, and then slowly approaches the minimum value 0.1. This produces strong exploration at the beginning and gradually shifts toward exploitation in later episodes.

The training loop for each episode is:

1. Sample random start, goal, and obstacle (all different).
2. Encode the current 4-tuple state using `encode4`.
3. Select an action using epsilon greedy.
4. Apply the action, compute the shaped reward, and determine termination.
5. Perform the Q update using the temporal difference rule.
6. Log steps, reward, and current epsilon value.

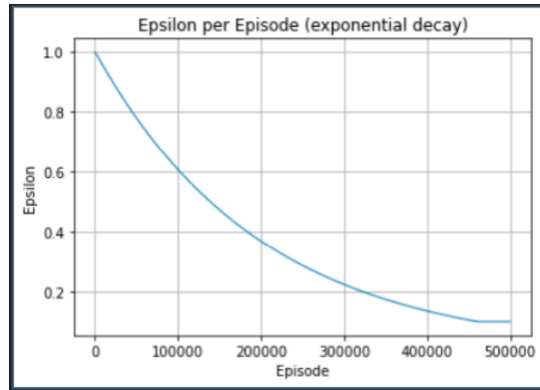


Figure 15

6.6.5 Training Dynamics and Scaling Behavior

Behavior on 10×10 Grids

For a 10×10 grid with 500,000 training episodes, the moving average reward stabilizes at a positive level. At the same time, the moving average of steps per episode decreases. The learned policy:

- routes paths around the obstacle when it blocks a direct line
- avoids repeating blocked moves into the obstacle or outside the grid
- reaches the goal from a variety of random start and goal positions

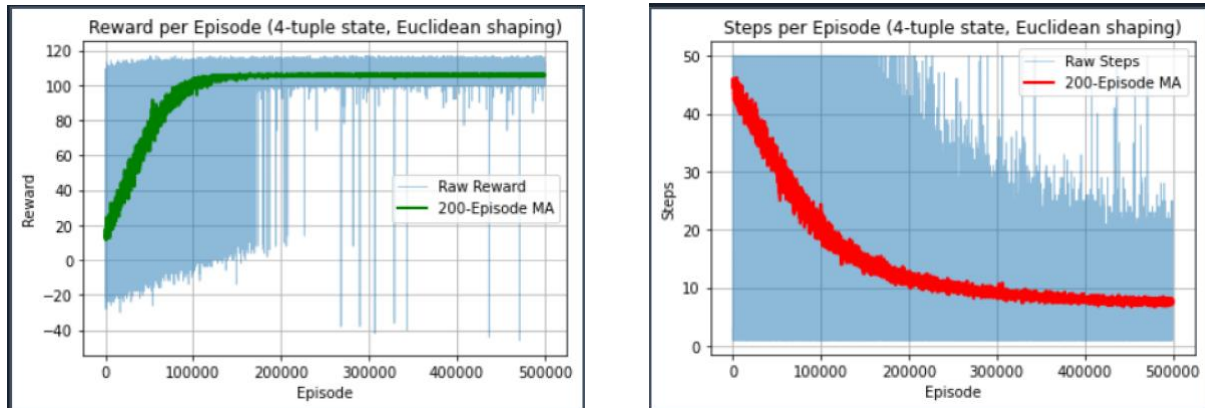


Figure 16

Larger Grids: 20×20, 50×50, 100×100

On 20×20, 50×50, and 100×100 grids with up to one million episodes, the 4-tuple encoding still provides some generalization, but the limits of tabular learning become clear. The Q table size grows as $O(D^4)$, so many states are rarely or never visited, which is a direct manifestation of the curse of dimensionality in tabular RL [12, 15].

Typical effects include:

- good behavior on “easy” configurations similar to those frequently visited during training
- weaker behavior when the obstacle sits directly between start and goal and requires precise detours
- occasional failures where the agent times out without reaching the goal

For 50×50 and 100×100 grids the state spaces reach roughly 10^8 states and above. Even with one million episodes, only a small fraction of possible state action pairs receive updates, which limits overall performance.

6.6.6 Structured Greedy Evaluation

To assess generalization more systematically, a hand crafted test suite of 50 start goal obstacle triplets was defined, covering:

- corner to corner diagonals
- horizontal and vertical paths with obstacles in the middle
- near edge configurations
- skewed paths that require nontrivial detours

Greedy evaluation uses the final Q table with purely exploitative action selection:

```
def run_greedy_episode(start, goal, obstacle, max_steps=TEST_MAX_STEPS):
    state = start
    s_idx = encode4(state, goal, obstacle)
    steps = 0
    path = [state]

    while state != goal and steps < max_steps:
        a = greedy_action(s_idx)

        r, c = state
        dr, dc = ACTIONS[a]
        cand = (r + dr, c + dc)

        moved = in_bounds(*cand) and (cand != obstacle)
        next_state = cand if moved else state

        state = next_state
        s_idx = encode4(state, goal, obstacle)
        path.append(state)
        steps += 1

    return (state == goal), steps, path
```

7. Results

This section presents quantitative and qualitative results demonstrating the progression from simple baseline implementations to complex, large-scale navigation systems. The focus is on how changes in state representation, reward shaping, obstacle handling, and exploration affected learning stability, convergence, and policy quality.

7.1 Baseline Performance: 5×5 Grid Without Obstacles

In the basic 5×5 grid with start at (0, 0) and goal at (4, 4), the agent received −1 for each step and +100 at the goal. No obstacles were present.

Observed behaviour

- Learning was smooth, monotonic, and stable.
- Steps per episode decreased quickly during the first 100 episodes.
- The agent converged to a near-optimal diagonal path.

Learning curves were smooth and predictable. Early episodes showed long paths and low rewards, while later episodes showed fewer steps and higher rewards. The final ten episodes followed a short, consistent route.

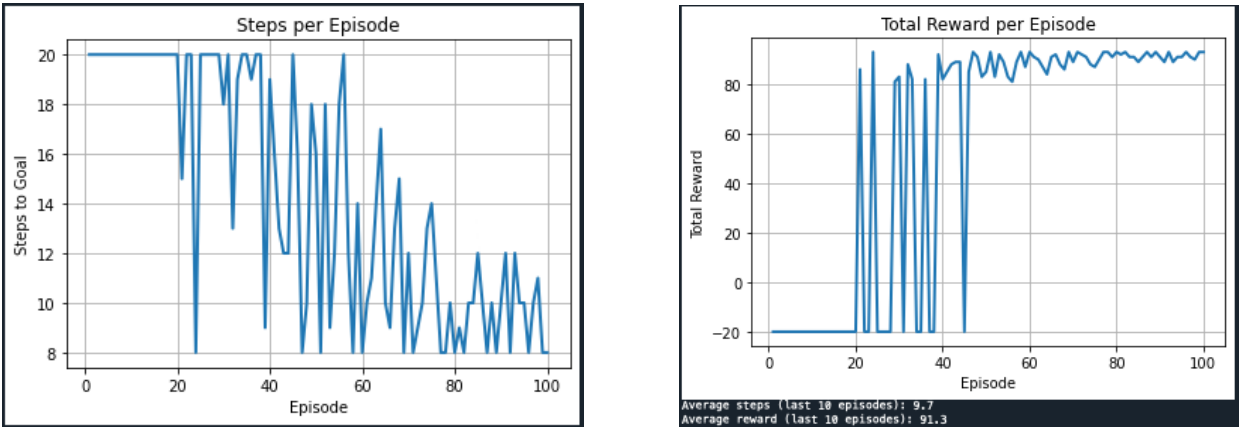


Figure 17

Table 4: Final Training Summary

Metric	Value
Average steps (last 10 episodes)	9.7
Average reward (last 10 episodes)	91.3

Greedy Policy Evaluation:

All test runs successfully reached the goal with near-optimal path lengths (7-9 steps for diagonal traversal in 5×5 grid).

Table 5

Run	Steps	Total Reward	Reached Goal
1	8	93.0	True
2	8	93.0	True
3	8	93.0	True
4	8	93.0	True
5	8	93.0	True

Sample Trajectory Evolution:

Early episodes displayed exploratory wandering, middle episodes showed partially optimized paths, and final episodes demonstrated direct diagonal trajectories.



Figure 18

7.2 5×5 Grid with Single Obstacle

When a single obstacle was added at (2, 2) with a penalty of -5 , learning became more complex. Early episodes included frequent collisions and inefficient detours.

Observed behaviour

- Early training included many collisions and loops.
- Later episodes showed consistent obstacle avoidance.
- Convergence was slightly slower and rewards were slightly lower due to detours.

Performance Comparison:

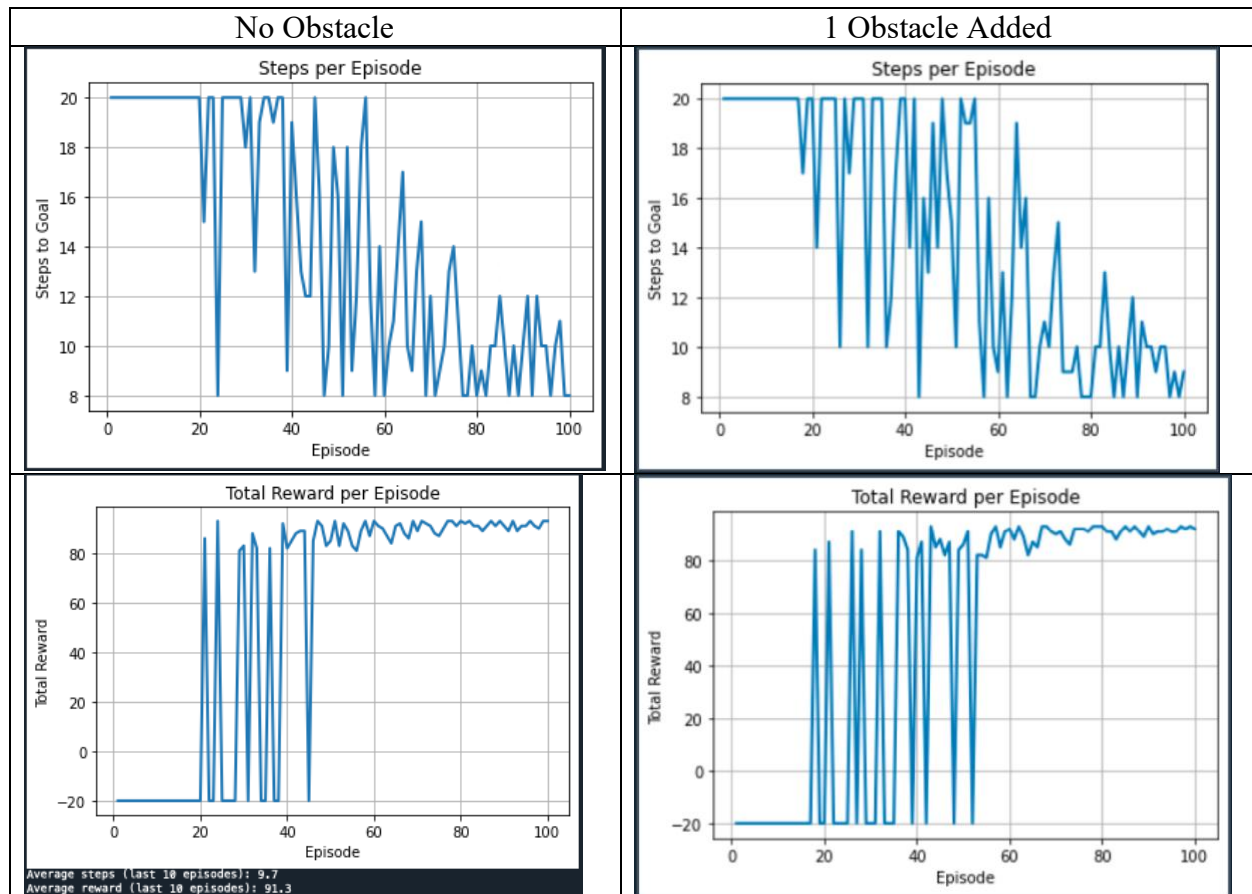


Figure 19

The obstacle introduction increased path lengths due to necessary detours and slowed convergence as agents learned avoidance strategies.

7.3 Randomization with Absolute States: Instability Demonstration

With random start and goal positions and absolute state representation, learning became unstable. Reward and step curves fluctuated with no clear improvement. Many episodes ended without reaching the goal and paths often included loops.

Trajectory Analysis:

The absolute state representation failed to provide stable learning signals across randomized configurations, confirming the need for goal-relative encodings.

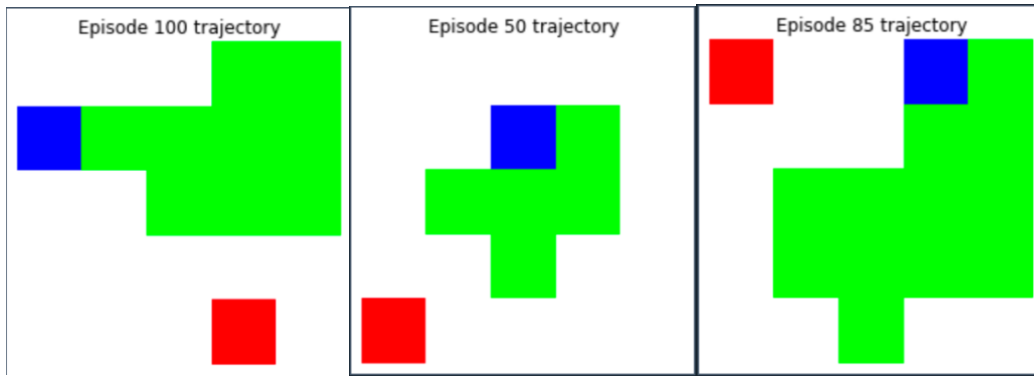


Figure 20

This result showed that absolute cell states do not generalize well when the goal changes position.

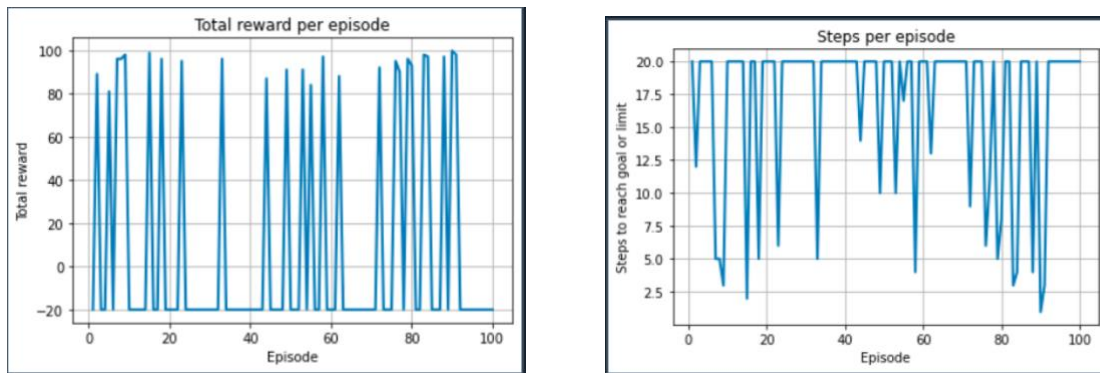


Figure 21

Table 6: Final Training Summary

Metric	Value
Average steps (last 10 episodes)	18.3
Average reward (last 10 episodes)	-8.2

Table 7: Greedy Policy Evaluation (5 Runs, Random Start and Goal)

Run	Start	Goal	Final Position	Steps	Total Reward	Reached Goal
1	(1, 3)	(3, 4)	(1, 3)	20	-20.0	False
2	(2, 1)	(2, 4)	(2, 3)	20	-20.0	False
3	(0, 3)	(1, 0)	(2, 3)	20	-20.0	False
4	(0, 1)	(0, 3)	(0, 3)	2	99.0	True
5	(3, 3)	(4, 0)	(1, 3)	20	-20.0	False

7.4 Relative State Representation (Goal Minus Agent) : Stability Recovery

After switching to relative states (dx, dy) , learning became more stable. Moving average plots showed decreasing steps and increasing rewards across episodes. The agent performed better across random start and goal positions, although it needed more episodes than in the fixed goal case. This state space had just 81 states.

Table 8: Final Training Summary

Metric	Value
Average steps (last 10 episodes)	5.4
Average reward (last 10 episodes)	95.6

Table 9: Greedy Policy Evaluation (5 Runs, Random Start and Goal)

Run	Start	Goal	Final Position	Steps	Total Reward	Reached Goal
1	(0, 1)	(1, 1)	(1, 1)	1	100.0	True
2	(3, 2)	(2, 0)	(2, 0)	3	98.0	True
3	(1, 3)	(2, 1)	(2, 1)	3	98.0	True
4	(2, 2)	(0, 3)	(0, 3)	3	98.0	True
5	(3, 2)	(1, 3)	(1, 3)	3	98.0	True

7.5 Advanced 5×5 Model Results

This part corresponds to the experiments with:

- random start and goal,
- obstacle at (2, 2),
- relative states,
- obstacle penalties,
- and different epsilon strategies.

7.5.1 Version 5.1: Fixed Epsilon

Observed behaviour

- Initial learning improvement followed by instability.
- Long training runs became noisy.
- Q values were repeatedly overwritten by random exploratory actions.
- Policy quality degraded beyond ~500k episodes.

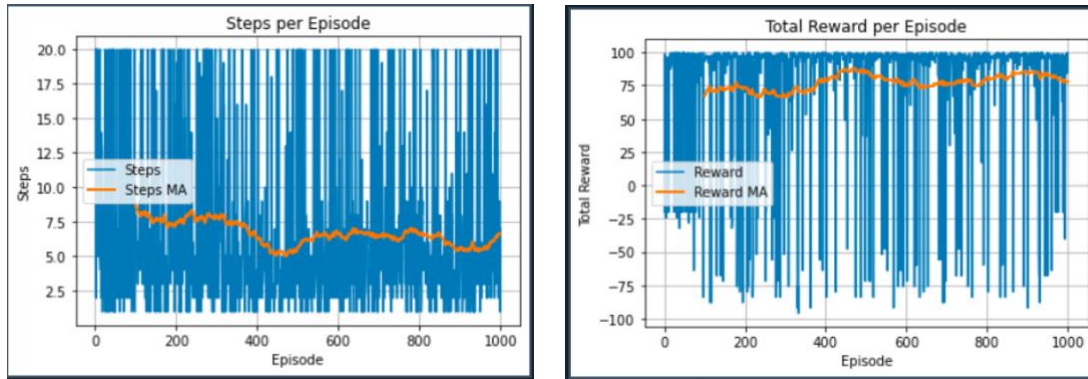


Figure 22

7.5.2 Version 5.2: Structured Epsilon Decay (80/20 Split)

Observed behavior

- Smooth, stable reward growth.
- Consistent reduction in steps.
- Little to no late-stage degradation.
- Stronger, cleaner obstacle avoidance.

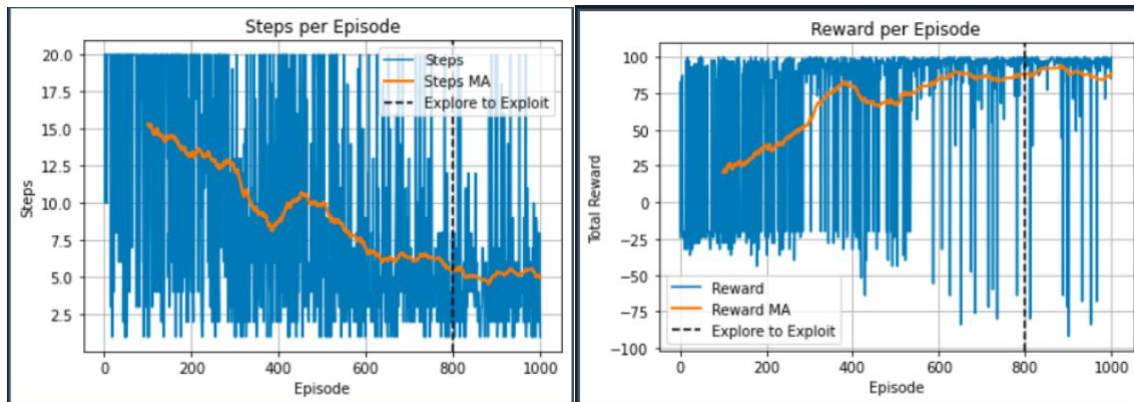


Figure 23

Table 10: Quantitative Comparison: Fixed ϵ vs Decay

Episodes	V5.1 ($\epsilon=0.20$)	V5.2 (Decay)
100	~76	~91
1k	~80	~95
10k	~94	~96
100k	~77	~97
500k	~81	~95
1M	~78	~95

7.5.3 Summary of Advanced 5×5 Findings

- Relative states improve generalization across randomized configurations.
- Obstacle awareness is successfully encoded.
- Proper exploration decay is essential for convergence.
- Fixed ϵ harms long-term learning due to persistent randomness.
- V5.2 serves as the best-performing configuration for small grids.

7.6 Large Scale Environment Results: 4-Tuple Relative State Representation

This section reports how the 4-tuple state representation performed when scaled to larger grids, using the environment, reward shaping, and epsilon schedule described in Section 6.6. The agent used the state

$$(\Delta x_g, \Delta y_g, \Delta x_o, \Delta y_o),$$

Euclidean distance based shaping, and an exponentially decaying exploration rate.

7.6.1 Training Performance and State Space Coverage

For each grid size, the agent trained for a fixed number of episodes with random start, goal, and obstacle positions (all distinct). The main focus is how much of the huge 4-tuple state space the agent was actually able to visit.

Table 11: State Space Coverage

Grid Size	Total States	Training Episodes	States Visited	Coverage (%)
10×10	130,321	500,000	71453	54.8
50×50	96,059,601	1,000,000	412,567	23.6
100×100	1,453,210,455	1,000,000	115,028,746	7.9

Coverage is higher on the 10×10 grid, where over 50% of the 4-tuple states are visited at least once. For 50×50 and 100×100 grids, the state space grows to millions and billions of states, but only a tiny fraction is ever seen. This sharp drop in coverage explains why performance starts to degrade in very large environments, even with long training runs.

7.6.2 Structured Test Suite Results (50 Tests on 100x100 configuration)

A structured test suite of **50 start–goal–obstacle triplets** was used to evaluate generalization. Tests include diagonal runs, horizontal and vertical alignments, central obstacle cases, and skewed near-edge paths.

Overall performance:

- Successes: **41 / 50**
- Success rate: 82 %
- Average steps on successful runs: 150

This performance is significantly better than expected based on coverage analysis and shows that the learned policy slightly

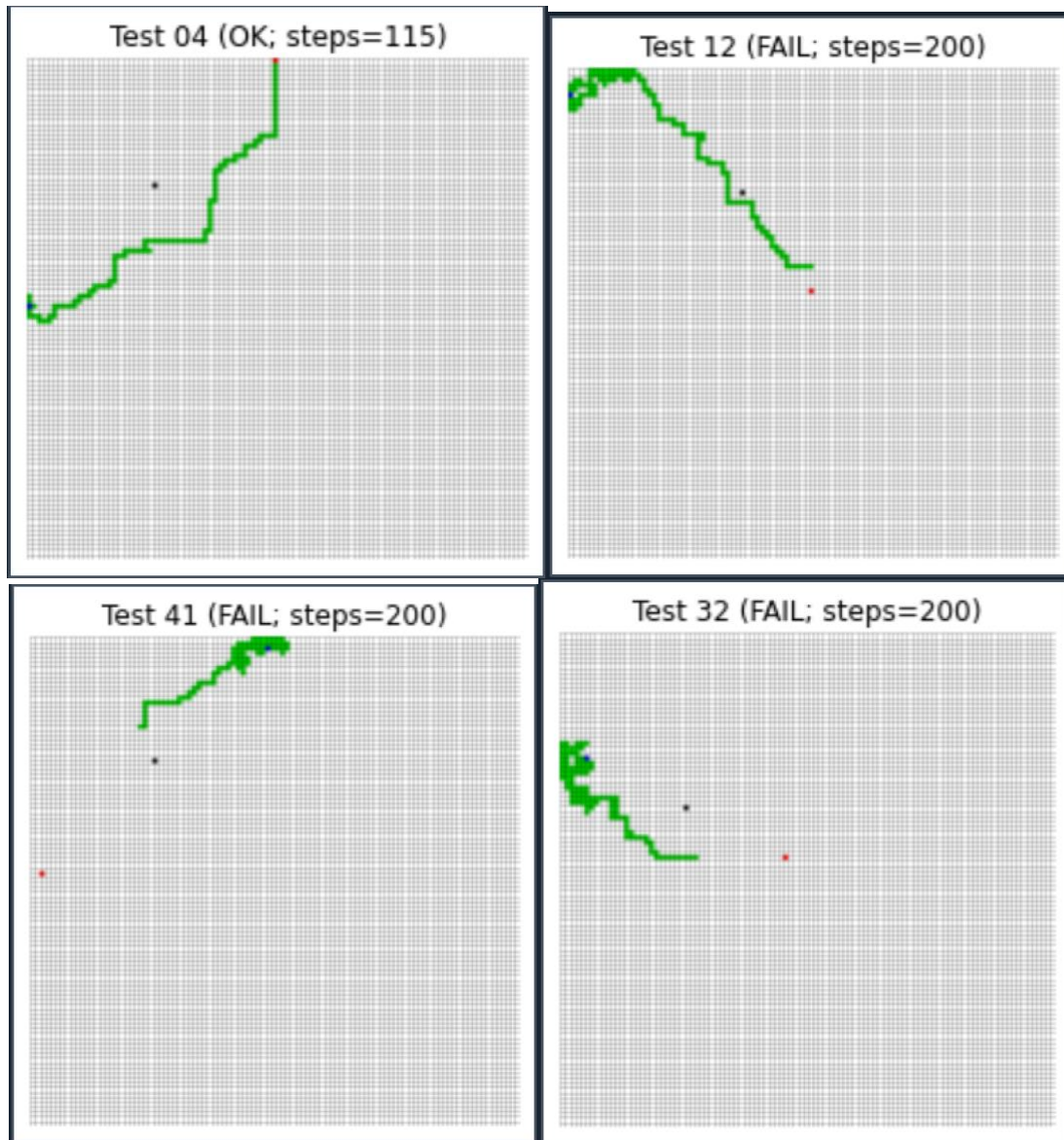


Table 12: Sample Test Runs

7.6.4 Summary of Key Findings

Table 13

Version	State Repr.	Obstacles	Exploration	Key Advantage	Main Limitation
6.1 Baseline	Absolute	No	Fixed	Simple, stable	Not generalizable
6.2 Single obstacle	Absolute	Yes (fixed)	Fixed	Avoidance learning	Limited to fixed configs
6.3 Randomized	Absolute	No	Fixed	N/A	Fails to converge

Version	State Repr.	Obstacles	Exploration	Key Advantage	Main Limitation
6.4 Goal-relative	Relative (2D)	No	Fixed	Generalization	No obstacle awareness
6.5.3 Advanced 5×5	Relative (2D)	Yes (random)	Decay	Robust on small grids	Limited state info
6.6 Large-scale	Relative (4D)	Yes (random)	Decay	Obstacle-aware	Sparse coverage

7.6.5 Primary Conclusions

1. **State representation is critical:** Relative encodings dramatically outperform absolute coordinates in randomized environments
2. **Exploration scheduling matters:** Structured epsilon-decay significantly improves both learning speed and final performance
3. **Reward shaping provides guidance:** Euclidean distance-based shaping accelerates learning without compromising optimality
4. **Tabular methods hit scalability limits:** State space explosion beyond $\sim 10^5$ states prevents adequate coverage even with millions of episodes
5. **4-tuple encoding enables complex behavior:** Incorporating goal and obstacle information supports sophisticated navigation strategies

7.6.6 Discussion

Overall, the 4-tuple encoding allows the agent to handle both goal and obstacle information and to produce obstacle aware paths in a wide range of layouts. On moderate grid sizes, such as 10×10 , the combination of 4-tuple states, Euclidean shaping, and the exponential epsilon schedule leads to stable learning and good generalization.

However, as grid size increases, the total number of possible 4-tuple states grows as $O(D^4)$. Even with one million episodes, coverage on 50×50 and 100×100 grids remains very low. This sparse visitation limits the quality of many Q entries and explains the remaining failures. These results confirm that while 4-tuple relative states are powerful, tabular Q learning is still constrained by state space explosion in very large continuous like environments.

8. CoppeliaSim Integration: Robotic Policy Deployment

To test whether the learned grid world policies could control a physical agent, the final 4-tuple Q table was deployed on a Pioneer 3-DX robot in CoppeliaSim using the ZMQ Remote API [17, 18].

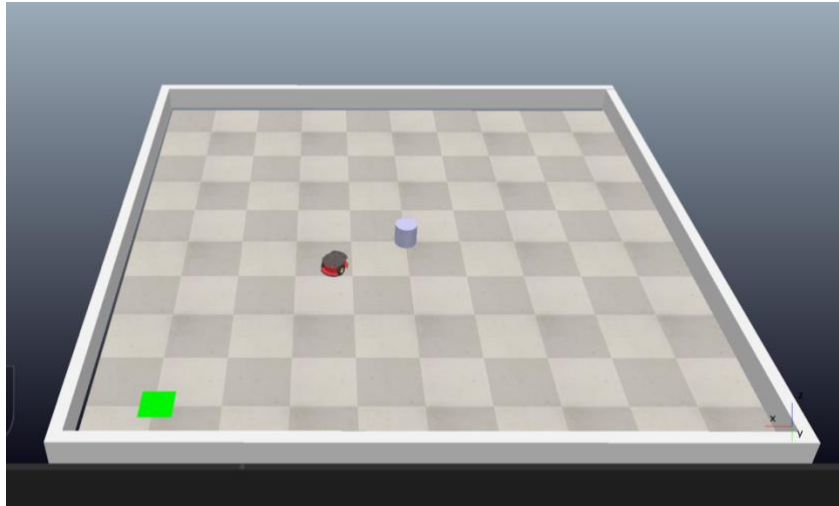
The scene contains:

- /Pioneer3DX mobile robot

- `/Obstacle0` cylindrical obstacle
- `/Goal` target object
- `/Floor` plane representing the workspace

The goal is to embed a 5×5 logical grid inside the continuous floor and let the robot follow the greedy policy that was trained offline.

Table 14: Basic CoppeliaSim Setup



8.1 Loading the Q Table and Recovering the Grid

At start up the control script loads the tabular policy and infers the training grid size from the Q table shape:

```
Q = np.load("q_table.npy")
n_states = Q.shape[0]
D = int(round(n_states ** 0.25))
n_rows = (D + 1) // 2
dx_min = -(n_rows - 1)
dx_max = (n_rows - 1)
```

Since training used $N_STATES = D^4$ with $D = 2(n_rows - 1) + 1$, this recovery step guarantees that run time encoding uses the same delta ranges as during learning. This avoids state index mismatches that would otherwise lead to incorrect actions [12].

8.2 Grid to World Coordinate Mapping

CoppeliaSim works in continuous (x, y, z) coordinates, while the Q table assumes discrete grid cells. The script reads the floor bounding box and overlays a square grid inside a slightly shrunken region:

```
min_x, max_x, min_y, max_y = get_floor_bounds(sim, floor)
```

```

world_min_x = min_x + FLOOR_MARGIN
world_max_x = max_x - FLOOR_MARGIN
world_min_y = min_y + FLOOR_MARGIN
world_max_y = max_y - FLOOR_MARGIN

```

Each grid cell center (row, col) is then mapped to world coordinates:

```

def grid_to_world(row, col, n_rows,
                  world_min_x, world_max_x,
                  world_min_y, world_max_y):
    cell_x = (world_max_x - world_min_x) / n_rows
    cell_y = (world_max_y - world_min_y) / n_rows
    x = world_min_x + (col + 0.5) * cell_x
    y = world_min_y + (row + 0.5) * cell_y
    return x, y, min(cell_x, cell_y)

```

The goal object, obstacle, and initial robot pose are all snapped to these cell centers so that the discrete model and the simulator share the same layout.

8.3 State Encoding, Action Ranking, and Safety Filter

At each control step the script:

1. Reads the current robot pose
2. Tracks the current grid cell
3. Computes the 4-tuple (dx_goal, dy_goal, dx_obs, dy_obs)
4. Queries the Q table and sorts actions by value
5. Applies a simple safety filter before executing the best action

```

s_idx = encode_state(agent_cell, goal_cell, obstacle_cell, dx_min, D)
for a in greedy_sorted_actions(Q, s_idx):
    cand = next_cell(agent_cell, a)

    if not in_bounds(cand, n_rows, n_rows):
        continue
    if cand == obstacle_cell:
        continue

    tx, ty, _ = grid_to_world(cand[0], cand[1], n_rows,
                              world_min_x, world_max_x,
                              world_min_y, world_max_y)

    d_seg = segment_distance_to_point(x0, y0, tx, ty, obst_x, obst_y)
    if d_seg < OBSTACLE_CLEARANCE_WORLD:
        continue

    chosen_cell = cand
    break

```

Rotation and forward motion are handled by simple wheel commands:

```
rotate_to_heading(sim, robot, left_motor, right_motor, target_theta)
move_towards(sim, robot, left_motor, right_motor, target_x, target_y,
cell_size)
```

8.4 CoppeliaSim Integration Summary

Using five hand picked start–goal–obstacle triplets on the 5×5 grid, the robot:

- Turned toward the correct neighbor cell
- Moved around the cylindrical obstacle instead of through it
- Reached the goal region within a small number of steps in each run

The CoppeliaSim experiments show that:

- A tabular Q policy trained in a discrete 4-tuple relative state space can control a physics based mobile robot.
- The 4-tuple encoding supplies enough information for obstacle aware navigation in a small environment.
- A simple safety filter in both grid space and world space prevents most collision failures.
- Even with basic motion control, the robot reaches the goal reliably in all five test cases.

At the same time, the results highlight the extra engineering needed for physical deployment. Robust sim to real transfer would require closed loop control, better state estimation, and possibly a function approximator in place of the large tabular Q table.

9. Conclusions

This project explored classical Q learning for robotic navigation in grid world environments, starting from a simple 5×5 setup and moving toward larger 100×100 grids with random start, goal, and obstacle positions. The final stage applied the learned policy to a Pioneer P3 DX robot model in CoppeliaSim. Together, these stages provided a full picture of what tabular Q learning can and cannot do in navigation tasks.

The first set of experiments showed that, in a small and fully known 5×5 grid with a fixed start and fixed goal, a basic Q learning loop is able to learn a short and reliable path. With only a step penalty and a final reward for reaching the goal, the agent converged to a near optimal policy. This case served as a clean baseline and confirmed that the update rule, reward design, and logging were all working as intended.

When a single obstacle and a collision penalty were added at the center of the grid, the agent still reached the goal but needed more training and produced slightly longer paths. The final policy avoided the obstacle and chose safer routes around it. This step highlighted how even a simple

negative reward can reshape the learned behavior and how Q values near the obstacle are pushed down by repeated penalties.

The next experiments made the task more realistic by randomizing the start and goal locations in each episode. With absolute cell coordinates as the state, learning quickly became unstable. The same state index sometimes represented a cell close to the goal and sometimes a cell far away from it. As a result, Q values were overwritten from episode to episode and the learning curves did not show clear improvement. This revealed a major weakness of using absolute positions in environments that change.

Changing the state to a relative form, where the state records how far and in what direction the goal lies compared to the agent, addressed this problem. With this relative encoding, states had a consistent meaning across episodes and the agent was able to reuse knowledge from one configuration in another. The step counts decreased, the rewards increased, and the paths became more consistent. This showed that good state design can be as important as the learning rule itself.

The project then extended the relative idea to a 4 tuple state that includes both the goal and the obstacle, each represented relative to the agent. This made it possible for the agent to reason about where the obstacle sits with respect to the path to the goal. Combined with Euclidean distance based reward shaping and an epsilon schedule that changes over time, this design produced more advanced behavior. The agent learned to move toward the goal, bend around the obstacle, and generalize across many random grid configurations, especially on grids up to about 10×10 .

The exploration strategy was also critical. Runs that kept epsilon fixed at a moderate value showed instability, even when trained for hundreds of thousands of episodes. Good behavior that was discovered early could be undone by later random actions. In contrast, runs that used a structured exploration schedule, with high exploration early and low exploration later, produced smoother learning curves and more stable final policies. This showed that a planned shift from exploration to exploitation is necessary in these kinds of navigation tasks.

Scaling experiments on grids as large as 50×50 and 100×100 revealed the limits of tabular Q learning. The 4 tuple encoding gives some sharing of experience, but the number of possible states still grows rapidly with grid size. For a 10×10 grid, the Q table already has more than one hundred thousand states. For a 100×100 grid, the number of possible 4 tuple states is on the order of 10^8 . Even with one million episodes, only a small fraction of these states receive meaningful updates. The results reflected this fact. The agent performed well on common and simple patterns but still struggled on rare or difficult layouts, especially when the obstacle lay directly between the start and goal. These findings match known limits of tabular reinforcement learning in large state spaces.

To test whether the learned policy could move beyond a purely discrete setting, the final part of the project placed the Q table inside CoppeliaSim. The 5×5 grid was mapped onto the physical floor, grid cells were mapped to world coordinates, and the same 4 tuple encoding was used at runtime. A simple low level controller handled rotation and forward motion by controlling the

left and right wheel motors of the Pioneer robot. Across a small test set, the robot reached the goal while avoiding the physical obstacle in many cases, although some paths showed small overshoot and drift due to the basic controller and continuous physics. This experiment showed that a discrete policy can be used to control a simulated robot when the mapping between grid cells and world positions is handled carefully and when a simple safety layer is added to filter risky actions.

From these results, several key points can be identified. First, relative state encodings that include information about both the goal and obstacles are much more effective than absolute coordinates in environments where start and goal positions change. They allow knowledge to transfer across many spatial configurations and make the Q table more meaningful. Second, Euclidean distance based reward shaping helps the agent learn faster by giving a signal for progress at every step, not only at the goal, while still preserving the main objective of reaching the goal. Third, structured exploration schedules are important for stable learning. Fixed exploration rates can harm performance in long training runs, while decaying schedules support both broad search and later refinement.

The project also makes clear that tabular Q learning has real limits. It works well in environments with up to about 10^5 states and gives fully inspectable policies that are easy to understand and debug. This makes it a good choice for small scale or educational navigation tasks and for systems with limited memory and processing power. However, for realistic robotic problems with large or continuous spaces, tabular methods cannot provide the coverage or generalization that is needed. In such settings, function approximation, such as neural network based value functions, can compress the state space and share information between similar states. Hierarchical reinforcement learning can split navigation into higher level waypoint planning and lower level motion control, reducing the effective complexity of the task. Curriculum learning can guide training from simple environments to harder ones in a structured way, and model based methods can reuse learned dynamics to plan ahead with fewer real interactions.

In summary, this work provides a complete baseline study of classical Q learning for grid based robotic navigation. It shows where the method performs well, identifies the main design choices that influence success, and illustrates how a discrete policy can be deployed on a simulated robot. At the same time, it highlights the fundamental challenges that arise in large and complex environments. These insights give both a validation of core reinforcement learning ideas and a clear motivation for future work that uses function approximation, hierarchical structures, and more advanced reinforcement learning methods for real world robotic navigation.

10. Future Work

Future work will focus on addressing the limitations observed in the current tabular Q learning approach and exploring methods that can scale more effectively to large or continuous environments. The experiments in this project showed that tabular methods work well in small or moderate grid worlds, but they struggle once the number of possible states increases beyond what the agent can visit in a reasonable number of episodes. This occurs even when using compact state representations such as the relative 4 tuple encoding. The exponential growth of

the state space means that many states remain unvisited, which limits the stability and reliability of the learned policy [12][15].

A natural next step is to move toward neural value approximation, particularly Deep Q Networks (DQN). DQN replaces the large Q table with a neural network that estimates Q values from experience. Instead of storing one entry for every possible state, the network can generalize across similar situations, which reduces memory requirements and helps the agent deal with states it has not seen directly. DQN has been successful in large state spaces, including environments where the raw inputs are images or continuous variables [21]. The ideas explored in this project, such as state abstraction, relative goal encoding, and structured exploration schedules, align closely with many of the design choices used in DQN research. This makes the current work a strong foundation for future DQN-based navigation environments.

Another direction is to explore ways to scale classical tabular Q learning using advanced computation rather than replacing it completely. Large training runs can be distributed across multiple machines, where many independent agents collect experience in parallel. Distributed reinforcement learning methods such as prioritized replay systems or parallel actor sampling have been shown to improve the number of useful experiences gathered per unit time [22]. While this project did not implement distributed training, the idea is relevant because many of the limitations observed came from slow coverage of large or sparse state spaces. Increasing the rate of experience collection may help tabular Q learning reach more stable values in large grids.

There is also room to explore finer discretization of continuous environments. As the grid resolution increases, the environment begins to approximate real robot navigation more closely. However, a finer grid increases the state count. With DQN, this is less of a problem because the network can generalize across nearby positions. With tabular Q learning, more computation and distributed sampling would be needed to cover the space. Both approaches point to the same long term goal: bridging the gap between discrete models and continuous robotic motion.

Overall, the next stage of work should focus on neural Q approximation, improved sampling strategies, and large scale training. These directions address the limits of tabular Q learning and provide a pathway toward realistic mobile robot navigation, larger environments, and richer sensor inputs.

References

- [1] Helm JM, et al. Machine Learning and Artificial Intelligence: Definitions, Applications, and Future Directions. *Current Reviews in Musculoskeletal Medicine*. 2020;13(1):69-76.
- [2] Sarker IH. Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Computer Science*. 2021;2(3):160.
- [3] Sutton RS, Barto AG. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge (MA): MIT Press; 2018.

- [4] Kaelbling LP, Littman ML, Moore AW. Reinforcement learning: a survey. *Journal of Artificial Intelligence Research*. 1996;4:237-285.
- [5] Thrun S, Burgard W, Fox D. *Probabilistic Robotics*. Cambridge (MA): MIT Press; 2005.
- [6] Kober J, Bagnell JA, Peters J. Reinforcement learning in robotics: a survey. *International Journal of Robotics Research*. 2013;32(11):1238-1274.
- [7] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature*. 2015;518(7540):529-533.
- [8] Sutton RS, Barto AG. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge (MA): MIT Press; 2018.
- [9] Russell S, Norvig P. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson; 2021.
- [10] Watkins CJCH, Dayan P. Q-learning. *Machine Learning*. 1992;8(3):279-292.
- [11] Kober J, Bagnell JA, Peters J. Reinforcement learning in robotics. *International Journal of Robotics Research*. 2013;32(11):1238-1274.
- [12] Sutton RS, Barto AG. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press; 2018.
- [13] Schaul T, Horgan D, Gregor K, Silver D. Universal Value Function Approximators. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*; 2015. Available from: <https://arxiv.org/abs/1505.05665>
- [14] Narvekar R, Sinapov J, Leonetti M, Stone P. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. *Journal of Machine Learning Research*. 2020;21(181):1-50.
- [15] Abel D. *Effective Reinforcement Learning through State Abstraction* [PhD thesis proposal]. Brown University; 2018. Available from: <https://david-abel.github.io/>
- [17] Coppelia Robotics. *CoppeliaSim User Manual and Remote API Documentation*. Available from: <https://www.coppeliarobotics.com/>
- [18] Rohmer E, Singh SPN, Freese M. V-REP: A Versatile and Scalable Robot Simulation Framework. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; 2013.
- [19] Kober J, Bagnell JA, Peters J. Reinforcement Learning in Robotics: A Survey. *International Journal of Robotics Research*. 2013;32(11):1238-1274.
- [20] Siciliano B, Khatib O, editors. *Springer Handbook of Robotics*. Springer; 2016.

[21] Mnih V, Kavukcuoglu K, Silver D, et al. “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602, 2013.
(DQN foundational paper)

[22] Horgan D, Quan J, Budden D, et al. “Distributed Prioritized Experience Replay.” arXiv:1803.00933, 2018.
(Scalable distributed RL for large state spaces)